

Week 1 Lecture Plan — Why Concurrency Exists (Pain First)

Improved Version

Pre-Class Preparation (Instructor)

Essential materials to prepare:

- ☐ Working demo server code (single-threaded + multi-threaded versions)
- ☐ Load testing client (pre-configured for 10, 50, 100 concurrent clients)
- ☐ Monitoring dashboard or terminal setup showing CPU/latency in real-time
- ☐ Pre-recorded backup demos (in case live demo fails)
- ☐ Lab 1.1 starter code repository ready for distribution

Room setup:

- Projector showing: terminal output + system monitor side-by-side
- Backup laptop for demos (Murphy's Law)

Learning Targets (Concrete & Measurable)

By the end of Week 1, students must be able to:

1. **Explain** the difference between latency and throughput with a concrete example
2. **Predict** when adding threads will help vs hurt (given a workload profile)
3. **Measure** both metrics using basic tooling and report them accurately
4. **Diagnose** why a server feels slow despite low CPU usage
5. **Articulate** the “waiting problem” that concurrency solves

Assessment checkpoint: Exit ticket (5 questions, 2 minutes)

Enhanced Structure: Two-Session Format (Recommended)

Session 1 (2h) — “The Pain & The Problem”

0) Hook + Course Context (5 min) Opening question (silent thinking):

“Why does your phone browser load 10 tabs simultaneously, but your backend can only handle requests one at a time?”

Course promise callback:

“By Week 14, you’ll diagnose production failures. Today, we learn why systems fail under load.”

Session roadmap (on board):

Today’s journey:

1. Watch a server die
2. Understand why it died
3. Fix it (partially)
4. Watch it die again (differently)

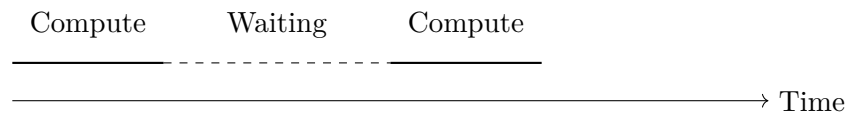
Conceptual Model: Why Waiting Exists Every program alternates between:

- CPU computation
- Waiting (I/O, network, disk, locks)

If a program waits 90% of the time, then:

The CPU is idle 90% of the time.

Concurrency exists to exploit idle waiting time.



1) Pain Demo Part 1: The Queue Forms (20 min) Demo setup (transparent to students)
Show the code first (2 minutes):

```
// Single-threaded server (simplified)
while (true) {
    Request req = acceptConnection();
    processRequest(req); // includes Thread.sleep(100) to simulate DB call
    sendResponse(req);
}
```

Important clarification: The 100ms sleep simulates a blocking database or external API call. The CPU is idle during this time.

Ask students immediately:

Is the server slow because the CPU is weak?

Live Demo Sequence

- **Step 1: Baseline (1 client)**
 - Send 1 request → response time $\approx 100\text{ms}$
 - CPU usage: $\approx 0\%$
 - Compute time: $\approx 1\text{--}2\text{ms}$
 - Waiting time: $\approx 98\text{ms}$

Board calculation:

$$\text{CPU Utilization} = \frac{\text{Compute Time}}{\text{Total Time}} = \frac{2}{100} = 2\%$$

Punchline: Server is idle 98% of the time.

- **Step 2: Light load (5 concurrent clients)**
 - All arrive at $t = 0$
 - Sequential handling
 - Latency $\approx 5 \times 100\text{ms} = 500\text{ms}$
 - CPU still $\approx 2\%$

Ask: Why does latency multiply if CPU is not busy?

- **Step 3: Breaking point (50 concurrent clients)**

- Latency $\approx 50 \times 100\text{ms} = 5000\text{ms}$
- CPU still mostly idle

Punchline: This is **I/O-bound starvation**. The CPU is free. The architecture is wrong.

Instructor-Led Mathematical Analysis

We model the system formally.

Each request requires:

$$T_{\text{compute}} = 2\text{ms}$$

$$T_{\text{wait}} = 98\text{ms}$$

Total:

$$T_{\text{service}} = 100\text{ms}$$

Arrival burst:

$$\lambda = 50 \text{ requests at } t = 0$$

Sequential server $\rightarrow$ behaves like M/M/1 with blocking.

Little's Law intuition:

$$L = \lambda W$$

If service time $W = 5s$ under burst load:

$$L = 50$$

All 50 requests are stuck inside the system.

Timeline Visualization

(draw TikZ as you already prepared)

Critical Thinking Question

What percentage of total time is productive computation?

$$\frac{2}{100} = 2\%$$

Follow-up question: If CPU is idle 98% of the time, what is the real bottleneck?

Expected answer: Blocking I/O.

Mini In-Class Exercise (3 min)

If sleep becomes 200ms instead of 100ms:

1. What happens to latency for 50 clients?
2. What happens to CPU utilization?

Expected reasoning:

Latency:

$$50 \times 200ms = 10s$$

CPU utilization:

$$\frac{2}{200} = 1\%$$

Doubling wait time halves CPU utilization.

Conclusion: The system is not CPU-bound. It is waiting-bound.

Transition Statement

If the CPU is idle 98% of the time, why do we allow it to serve only one request?

This is the birth of concurrency.

2) Conceptual Foundation (30 min) **Goal:** Give students the vocabulary + mental models to explain what they just observed.

Concept 1: Latency vs Throughput (10 min)**Warm-up prompt (1 min):**

A system can be “fast” in two different ways. What are they?

Interactive exercise (5 min):

- **Server A:** completes each request in 50ms, but can only handle 10 requests/second.
- **Server B:** completes each request in 5000ms, but can also handle 10 requests/second.
- **Question:** Which server would you rather use as a human? Why?

Expected student insight: Humans care first about **latency**. Businesses often care about **throughput**.

Definitions (emerge from answers):

- **Latency:** time to finish *one* request (user-facing).
- **Throughput:** requests completed per second (system/business metric).

One-line reality check:

A system can have high throughput while still feeling slow (high latency).

Micro-example (board, 1 min): If a single-threaded server takes 100ms per request, then:

$$\text{Throughput}_{max} \approx \frac{1}{0.1} = 10 \text{ req/s}$$

But under burst load, latency grows linearly.

Visual aid: Restaurant analogy (2 min):

- **Fast food:** low latency, high throughput
- **Fine dining:** high latency, low throughput
- **Festival food truck:** throughput stays high, but lines create high latency (what we just saw)

Checkpoint question (30 sec):

Which metric does a user feel immediately: latency or throughput?

Concept 2: Concurrency vs Parallelism (10 min)**Definitions:**

- **Concurrency:** managing many tasks at once (interleaving, scheduling, structure).
- **Parallelism:** executing tasks at the same time (requires multiple cores).

Critical insight (say it slowly):

Concurrency is about **structure**. Parallelism is about **execution**.

Mini-demo (2 min): Show Task Manager / htop:

- Many threads exist
- Only a few run at any instant
- Scheduler switches among them (illusion of simultaneity)

Pair exercise (2 min):

A single-core CPU running 10 threads: concurrent, parallel, both, or neither?

Expected answer: Concurrent only (interleaving, not simultaneous execution).

Instructor clarification (1 min):

- One core $\Rightarrow$ at most one thread runs at a time $\Rightarrow$ no true parallelism.
- But concurrency can still improve responsiveness when tasks spend time waiting.

Concept 3: I/O Wait vs CPU Work (10 min)**The ratio that matters:**

$$T_{service} = T_{cpu} + T_{io}$$

Typical profiles (context, not memorization):

- **I/O-bound:** web servers, databases, file services (often 80–95% waiting)
- **CPU-bound:** video encoding, ML training, large crypto operations

Guiding question (2 min):

If 90% of request time is I/O wait, what happens when you add 8 CPU cores?

Expected answer: Almost nothing for that request, because extra cores do not reduce waiting time.

Light theoretical hook (Amdahl preview, 2 min): If only 10% is CPU work (parallelizable part is limited), then speedup has a hard ceiling:

$$S_{max} \approx \frac{1}{0.9} \approx 1.11$$

Even “infinite cores” cannot make waiting disappear.

Closing statement (tie back to demo):

Concurrency helps most when systems spend most of their time waiting.

Transition question (to next section):

So what is the simplest way to overlap waiting time for many clients?

(Expected: threads / async I/O / event loop.)

Theoretical Framing: Why Concurrency Works (and Fails)

Practical systems fail for mathematical reasons.

Before adding threads blindly, we must understand three foundational principles:

- Amdahl's Law (Limits of Speedup)
- Little's Law (Queue Behavior)
- Context Switching Cost (Hidden Overhead)

1) Amdahl's Law — The Limit of Parallel Speedup Amdahl's Law states that the maximum theoretical speedup of a program is limited by the fraction that must run sequentially.

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Where:

- P = parallelizable fraction
- N = number of threads/processors
- $S(N)$ = maximum speedup

Example:

Suppose:

- 80% of the server work is waiting or parallelizable ($P = 0.8$)
- 20% must remain sequential

With 10 threads:

$$S(10) = \frac{1}{0.2 + \frac{0.8}{10}} = \frac{1}{0.2 + 0.08} = \frac{1}{0.28} \approx 3.57$$

Even with 10 threads, the maximum speedup is only $3.57\times$.

Important: Even infinite threads cannot exceed:

$$\frac{1}{1 - P}$$

In this case:

$$\frac{1}{0.2} = 5$$

Theoretical upper bound = $5\times$ speedup.

Concurrency improves performance, but mathematics imposes a ceiling.

2) Little's Law — Why Queues Form Little's Law relates:

$$L = \lambda W$$

Where:

- L = average number of requests in system
- λ = arrival rate (requests per second)
- W = average time in system

Example:

If:

- 50 requests per second arrive ($\lambda = 50$)
- Each request takes $200\text{ms} = 0.2\text{s}$

Then:

$$L = 50 \times 0.2 = 10$$

There will be 10 concurrent requests inside the system on average.

If you only allow 1 thread:

Queue grows immediately.

Little's Law explains:

High latency automatically increases queue size.

Concurrency reduces waiting time by overlapping execution.

3) Context Switching Cost — Why Threads Hurt Threads are not free.

Each context switch involves:

- Saving registers
- Flushing CPU pipelines
- Possible cache invalidation

Assume:

- Context switch cost = 5 microseconds
- 100,000 switches per second

Overhead:

$$100,000 \times 5\mu\text{s} = 500,000\mu\text{s} = 0.5\text{s}$$

Half the CPU time is wasted switching!

Too many threads reduce throughput.

Concurrency helps until scheduling overhead dominates.

Synthesis

- Amdahl: Parallelism has a limit.
- Little: Waiting increases queue size.
- Context switching: Too many threads reduce performance.

Concurrency is a balance between:

Idle time exploitation vs Scheduling overhead.

Engineers must measure before optimizing.

Concrete example:

- 90% parallelizable, infinite cores $\Rightarrow$ max speedup = $1/0.1 = 10\times$

Student calculation:

“If 50% is parallelizable, what’s the max speedup with 100 cores?”

Expected answer: $\approx 2\times$

Key takeaway (write on board):

“You cannot parallelize waiting away. You must hide it.”

3) Transition Activity: Predict–Then–Test (15 min) Scenario: “We’re about to add threads. Predict what will happen to:”

1. Latency (p50)
2. Throughput
3. CPU usage

Students write predictions (1 min).

Reveal answer in Session 2.

4) Common Misconceptions Quiz (10 min) True/False (with defense in pairs):

1. “High throughput always means low latency.” (False)
2. “Adding more threads always improves performance.” (False)
3. “Parallelism requires multiple cores; concurrency does not.” (True)
4. “If CPU is at 10%, the system is underutilized.” (Depends)
5. “Concurrency is only useful for I/O-bound tasks.” (Mostly true; nuanced)

5) Session 1 Wrap + Homework Bridge (10 min) Summary (students repeat back):

- Concurrency $\neq$ speed; concurrency = hide waiting
- Latency $\neq$ throughput (both matter)
- Most servers are I/O-bound

Before next session:

- Review the demo code (posted)
- Think: “How would you fix the waiting problem?”

Exit ticket (2 min):

1. Define latency in one sentence.
2. Why is CPU low when latency is high?
3. True/False: More threads always help.

Session 2 (2h) — “The Fix & The New Problem”

0) Recap + Answer Reveal (5 min) Revisit predictions from Session 1 and compare with measured outcomes.

1) Pain Demo Part 2: Multi-Threaded Server (25 min) Demo setup (show new code):

```
ExecutorService pool = Executors.newFixedThreadPool(10);

while (true) {
    Request req = acceptConnection();
    pool.submit(() -> {
        processRequest(req); // still 100ms sleep
        sendResponse(req);
    });
}
```

Key change: “Now 10 requests can wait simultaneously.”

Live demo sequence

- **Test 1: Same 50 clients**
 - Latency drops from ~5000ms → ~500ms
 - Throughput: ~10× improvement
 - CPU still low (still I/O-bound)
 - **Punchline:** “Hiding waiting works.”
- **Demo twist: Push to failure**
 - Test 2: 200 clients, pool size 10 → latency spikes again (queue forms)
 - Test 3: increase pool to 200 → initial improvement, then degradation

Example table to show:

Pool size	Latency p95	Throughput	CPU
10	500ms	100/sec	5%
50	200ms	250/sec	15%
200	300ms	200/sec	40% <- worse!
500	1000ms	150/sec	60% <- much worse!

Key realization:

“Threads are not free. There is a sweet spot.”

2) Analysis: Why Threads Helped (15 min) Timeline comparison (board)**Single-threaded:**

```
t=0      t=100   t=200   t=300   ...   t=4900
|req1|  |req2|  |req3|           |req50|
```

Multi-threaded (pool=10):

```
t=0      t=100   t=200   t=300   t=400   t=500
|req1|
|req2|
...
|req10|
|req11|
|req12|
...
```

Calculation:

- Single: $50 \times 100\text{ms} = 5000\text{ms}$ for last client
- Pooled (10): $5 \text{ batches} \times 100\text{ms} = 500\text{ms}$ for last client

Why it worked: overlapped waiting periods; used time previously idle.

3) Analysis: Why Too Many Threads Hurt (15 min) Sources of overhead

- **Context switching cost:** with hundreds of threads on few cores, scheduler thrashes
- **Memory overhead:** each thread has a stack; large thread counts waste memory
- **Contention:** threads compete for shared resources (locks, scheduler, queues)

Rule of thumb (write large):

$$\text{Optimal thread count} \approx \text{cores} \times \left(1 + \frac{\text{wait time}}{\text{compute time}} \right)$$

Example:

- $\text{wait}/\text{compute} = 90/10 = 9$
- $8 \text{ cores} \times (1 + 9) = 80 \text{ threads}$ (not 500)

4) Case Study: Redis & SQLite (15 min) Prompt: “Given what we learned, why choose single-threaded designs?”**Expected points:**

- simpler (no races, fewer locks)
- predictable latency
- cache locality
- single-threaded can still be extremely fast for in-memory workloads

Key insight:

“Engineering is tradeoffs. Single-threaded is a choice, not a limitation.”

5) Lab 1.1 Briefing (30 min) What students must submit:**(1) Measurements table:**

Load Level	Single-Thread			Multi-Thread (N=?)		
	p50	p95	Throughput	p50	p95	Throughput
Light (10)						
Medium (50)						
Heavy (100)						

(2) Concurrency scorecard:

- [] Thread-safe? (if shared state exists, is it protected?)
- [] Visibility guaranteed? (N/A for basic server)
- [] Deadlock-free? (N/A for basic server)
- [] Liveness guaranteed? (are requests eventually served?)
- [] Bounded resources? (did you cap thread pool? queue?)
- [] Failure recovery path? (timeouts? retries? or none?)

(3) Explanation paragraph (5–8 sentences) using rubric language:

- mechanism changed (threads)
- why performance improved (hiding I/O wait)
- tradeoff introduced (resource overhead, complexity)

Starter structure (show):

```
lab1/
|-- single_threaded_server.java
|-- multi_threaded_server.java
|-- client.java (load generator)
|-- README.md (how to run)
'-- measurements_template.md
```

How to run tests:

```
# Terminal 1: Start server
java SingleThreadedServer 8080

# Terminal 2: Run load test
java Client localhost 8080 --clients=50 --requests=10

# Output: CSV with latencies
```

Common pitfalls:

- Not warming up (JVM warmup): discard first 10–20 measurements
- Server and client on same machine: acceptable, but must be noted
- Thread pool size = client count: defeats the purpose; test multiple sizes (5, 10, 20, 50)

6) Hands-On Exercise: Predict Optimal Pool Size (10 min) Scenario:

- 8-core machine
- request profile: 90% I/O wait, 10% CPU
- target: 100 concurrent clients

Question: “What pool size would you try first? Why?”

Calculation together:

$\text{wait/compute} = 90/10 = 9$

optimal $\approx 8 \times (1 + 9) = 80$ threads

Try pool sizes: 20, 40, 60, 80

Measure and find the knee of the curve

7) Common Mistakes + Misconceptions (10 min)

- “More threads = faster” (false; demonstrated)
- “High throughput means good UX” (false; latency matters)
- “Parallelism and concurrency are the same” (false)
- “If CPU is low, the system is underutilized” (depends; I/O-bound is expected)

8) Session 2 Wrap + Next Week Preview (10 min) Summary (students state aloud):

- Threads hide waiting by overlapping I/O
- Too many threads introduce overhead
- Engineering is finding the sweet spot

Next week teaser:

“Thread pools work great... until they don’t. Next week: what breaks when threads share data.”

Final thought (board):

Concurrency is not speed.

Concurrency is hiding waiting.

But hiding waiting creates new problems.

Additional Materials Needed**1) Slide deck outline (12 slides max):**

1. Title + Course context
2. Learning targets
3. Demo 1 setup (code snippet)
4. Demo 1 results (clients vs latency)
5. Latency vs throughput (definitions + example)

6. Concurrency vs parallelism (definitions + visual)
7. Amdahl's law (formula + example)
8. Demo 2 setup (code snippet)
9. Demo 2 results (comparison table)
10. Why threads hurt (overhead sources)
11. Lab 1.1 deliverables checklist
12. Week 1 summary + Week 2 preview

2) Grading rubric for Lab 1.1:

Measurements table (30%):

- ☐ All cells filled (10%)
- ☐ Reasonable values (10%)
- ☐ Multiple load levels tested (10%)

Concurrency scorecard (20%):

- ☐ All items addressed (10%)
- ☐ Honest assessment (not all “yes”) (10%)

Explanation paragraph (40%):

- ☐ Mechanism described (10%)
- ☐ Performance change explained (10%)
- ☐ Tradeoff identified (10%)
- ☐ Uses rubric language (10%)

Code quality (10%):

- ☐ Runs without modification (5%)
- ☐ Reasonable structure (5%)

Total: /100

3) Pre-lab checklist (student-facing):

Before Lab 1.1:

- ☐ Reviewed demo code from lecture
- ☐ Java development environment set up
- ☐ Can run provided starter code
- ☐ Understand how to measure latency (p50/p95)
- ☐ Know how to monitor CPU usage

During Lab 1.1:

- ☐ Warm up runtime before measurements
- ☐ Test at least 3 different load levels
- ☐ Try at least 3 different thread pool sizes
- ☐ Record CPU observations for each test
- ☐ Fill scorecard honestly (not all boxes checked)

After Lab 1.1:

- ☐ Compare single vs multi-threaded results
- ☐ Identify optimal thread pool size
- ☐ Write explanation using rubric language
- ☐ Proofread before submission

Key Improvements Over Original Plan

1. Exit tickets + entrance tickets for formative assessment
2. Predict–then–test for active learning
3. Concrete calculations (Amdahl + pool sizing rule of thumb)
4. Failure demonstration (limits, not just success)
5. Rubric alignment from day 1 (quality language early)
6. Preparation checklist (reduces “how do I start” issues)
7. Memory aids (“Concurrency is hiding waiting” repeated)
8. Two-session split with cliffhanger to maintain engagement
9. Real-world anchors (Redis, restaurant analogy)
10. Explicit misconception destruction

Instructor Self-Checklist**Before Week 1:**

- ☐ Demo code tested on classroom machine
- ☐ Backup demos recorded
- ☐ Starter code repository accessible
- ☐ Grading rubric posted
- ☐ Exit ticket questions printed/digital

During Session 1:

- ☐ Students can see terminal + system monitor simultaneously
- ☐ Exit ticket collected

- ☐ Demo code shared immediately after class

During Session 2:

- ☐ Predictions revisited
- ☐ Lab deliverables crystal clear
- ☐ Students leave knowing how to start

After Week 1:

- ☐ Lab submissions graded with rubric
- ☐ Common mistakes noted for Week 2 opening
- ☐ Optimal thread pool sizes from student data collected (reuse in Week 5)